

Chapter 07 · Part 2

Runnables in LangChain — Part 2

The five Runnable primitives — Sequence, Parallel, Passthrough, Lambda, and Branch — that let you orchestrate any AI workflow, plus LCEL's declarative pipe syntax.

WHAT THIS CHAPTER COVERS

- ▶ RunnableSequence for chained execution
- ▶ RunnableParallel for concurrent branches
- ▶ RunnablePassthrough for pass-through data
- ▶ RunnableLambda for custom Python logic
- ▶ RunnableBranch for conditional routing
- ▶ LangChain Expression Language (LCEL)
- ▶ Real code using ChatGroq & HuggingFaceEndpoint
- ▶ Building multi-step, branching AI pipelines

Runnables in LangChain — Part 2

RECAP — CHAPTER 7, PART 1

Part 1 established *why* Runnables exist: LangChain's core components (Prompt, Model, Parser, Retriever) originally used inconsistent method names, making them hard to connect. LangChain standardized every component behind a common `Runnable` abstract base class with an `invoke()` method, and introduced a custom `RunnableConnector` to chain components sequentially — the output of one becoming the input of the next. Part 2 picks up from there and introduces LangChain's official, built-in tools for this orchestration: the five **Runnable Primitives**.

`RunnableSequence``RunnableParallel``RunnablePassthrough``RunnableLambda``RunnableBranch``LCEL Pipe Syntax`

1. Two Categories of Runnables

Before diving into the primitives, it is essential to understand that LangChain divides all Runnables into two distinct categories based on their purpose.

1.1 Task-Specific Runnables

These are the core LangChain components — **Prompt Templates, LLMs, Retrievers, Output Parsers** — that have been converted into Runnables so they can be plugged into pipelines. Each one still has its own independent purpose (a prompt template designs prompts, an LLM wrapper talks to a model), but because each also implements the `Runnable` interface, it can be composed with other Runnables.

`ChatOpenAI`, `PromptTemplate`, and any retriever are all examples: they are components *first*, and Runnables *as a side effect* of standardization.

1.2 Runnable Primitives

These are Runnables whose entire purpose is to **connect other Runnables together** so that complex AI workflows can be built. They do not process data in a domain-specific way (they don't generate text or embeddings) — instead, they define the *execution logic*: sequential, parallel, or conditional composition.

IMPORTANT

Task-specific Runnables answer “**what should happen**” at one step (generate a prompt, call an LLM). Runnable primitives answer “**how the steps relate to each other**” (in order, in parallel, or conditionally). This chapter focuses entirely on the five primitives: **`RunnableSequence`**, **`RunnableParallel`**, **`RunnablePassthrough`**, **`RunnableLambda`**, and **`RunnableBranch`**.

Category	Purpose	Examples
Task-Specific Runnable	Performs one specific job in the pipeline	PromptTemplate, ChatOpenAI, ChatGroq, StrOutputParser, Retriever
Runnable Primitive	Orchestrates how other Runnables interact	RunnableSequence, RunnableParallel, RunnablePassthrough, RunnableLambda, RunnableBranch

Quick Revision — Runnable Categories

- LangChain Runnables split into two categories: task-specific and primitives.
- Task-specific Runnables (Prompt, Model, Parser, Retriever) each have their own domain purpose.
- Runnable primitives exist purely to orchestrate/connect task-specific Runnables.
- Primitives define execution logic: sequential, parallel, or conditional.
- The five primitives covered in this chapter are Sequence, Parallel, Passthrough, Lambda, and Branch.

2. RunnableSequence

Definition

`RunnableSequence` is a Runnable primitive that chains two or more Runnables so they execute one after another, automatically feeding the output of each step as the input to the next.

Why It Exists

Sequential execution (Prompt → Model → Parser) is the single most common pattern in any LLM application. Instead of every engineer hand-writing a connector class, LangChain ships `RunnableSequence` as a built-in class that performs exactly this job.

How It Works

1. Instantiate `RunnableSequence`, passing every Runnable to be connected, in order.
2. There is no limit on how many Runnables can be chained — two or twenty.
3. Calling `.invoke()` on the resulting chain runs every step in order and returns the final step's output.

Example — Two-Hop Joke & Explanation Chain

The example below builds a six-step chain: a joke is generated about a topic using a HuggingFace-hosted Qwen/Qwen2.5-7B-Instruct model, and the resulting joke is immediately fed back through the same model to produce a plain-language explanation.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableSequence
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

# HuggingFaceEndpoint wraps a hosted HF model; ChatHuggingFace adapts
# it to the chat-model interface expected by the rest of LangChain.
model = ChatHuggingFace(
    llm = HuggingFaceEndpoint(
        repo_id = "Qwen/Qwen2.5-7B-Instruct"
    )
)

prompt1 = PromptTemplate(
    template = "Give me a joke about {topic}.",
    input_variables = ["topic"]
)

prompt2 = PromptTemplate(
    template = "Explain this joke to me in simple terms: {joke}.",
    input_variables = ["joke"]
)

parser = StrOutputParser()

# Six Runnables chained end-to-end: prompt1 -> model -> parser ->
# prompt2 -> model -> parser. Output of each step feeds the next.
chain = RunnableSequence(prompt1, model, parser, prompt2, model, parser)

result = chain.invoke({"topic" : "Freedom"})

print(result)

```

Each Runnable in the tuple is applied in order: `prompt1` builds the joke prompt, `model` generates the joke, `parser` extracts it as a plain string, `prompt2` wraps that string into an explanation prompt, `model` generates the explanation, and the final `parser` extracts the explanation string that gets printed.

`RunnableSequence(prompt1, model, parser, prompt2, model, parser)`



Figure 7.1 — Six Runnables connected end-to-end using `RunnableSequence`.

BEST PRACTICE

Reuse the same `model` and `parser` instances across multiple steps of a sequence instead of recreating them — as shown above, one `model` object is referenced twice in the same chain.

Quick Revision — `RunnableSequence`

- `RunnableSequence` chains `Runnables` to run one after another.
- Output of each `Runnable` automatically becomes input to the next.
- No limit on how many `Runnables` can be chained together.
- It is a built-in `LangChain` class — no custom connector code needed.
- `RunnableSequence` itself is a `Runnable`, so it can be nested inside other primitives.

3. `RunnableParallel`

Definition

`RunnableParallel` is a `Runnable` primitive that runs multiple `Runnables` concurrently against the **same input**, collecting all their outputs into a single dictionary.

Why It Exists

Some workflows need one piece of input processed in multiple independent ways at once — for example, generating both a tweet and a LinkedIn post from the same topic, potentially using different models best suited to each platform's style.

How It Works

1. Instantiate `RunnableParallel` with a dictionary whose keys are output labels and whose values are `Runnables` (often themselves a `RunnableSequence`).
2. Every branch receives the exact same input.
3. All branches execute independently and in parallel.
4. The final result is a dictionary keyed exactly as defined at construction time.

Example — Tweet & LinkedIn Post Generator

The code below feeds one topic into two independent branches, each its own `RunnableSequence`, using two different model providers — a HuggingFace-hosted Qwen model for the tweet, and a Groq-hosted Llama model for the LinkedIn post.

```

from langchain_core.runnables import RunnableParallel, RunnableSequence
from langchain_huggingface import HuggingFaceEndpoint, ChatHuggingFace
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

# Two different model providers power the two parallel branches.
model1 = ChatHuggingFace(
    llm = HuggingFaceEndpoint( repo_id = "Qwen/Qwen2.5-7B-Instruct" )
)

model2 = ChatGroq(
    model = "llama-3.3-70b-versatile",
)

prompt1 = PromptTemplate(
    template = "Generate a catchy/interesting tweet on the topic : {topic}",
    input_variables = ["topic"]
)

prompt2 = PromptTemplate(
    template = "Generate a catchy/interesting LinkedIn post on the topic : {topic}",
    input_variables = ["topic"]
)

parser = StrOutputParser()

# Dict keys become the output dict's keys; each value is its own
# independent RunnableSequence, executed concurrently.
parallel_chain = RunnableParallel(
    {
        "tweet": RunnableSequence(prompt1, model1, parser),
        "linkedin_post": RunnableSequence(prompt2, model2, parser)
    }
)

output = parallel_chain.invoke({"topic": "AI Engineering Career Potential"})

print(output["tweet"])
print(output["linkedin_post"])

```

`parallel_chain.invoke()` sends `{"topic": "..."}` to both branches simultaneously. Branch "tweet" runs `prompt1 → model1 → parser`; branch "linkedin_post" runs `prompt2 → model2 → parser`. The result is a single dict: `{"tweet": "...", "linkedin_post": "..."}` .

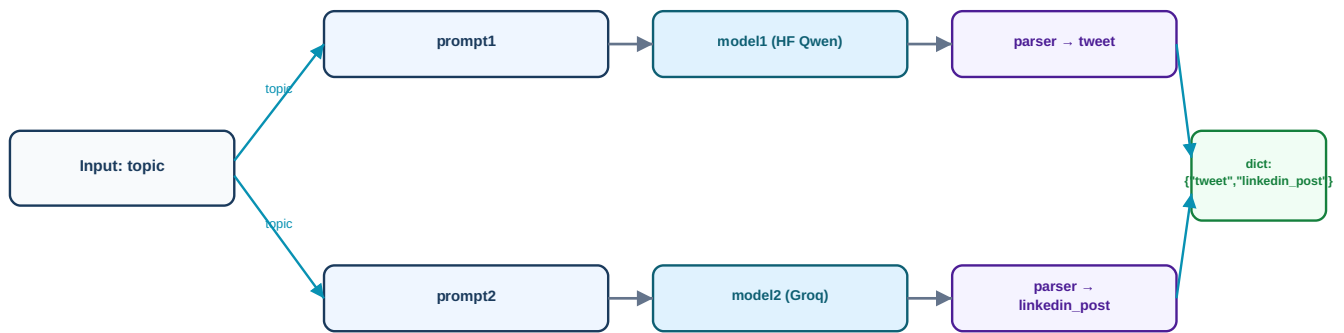


Figure 7.2 — `RunnableParallel` fanning one input into two independent branches with different model providers.

INTERVIEW TIP

Be ready to explain that `RunnableParallel`'s output is always a dictionary, and that each value can itself be a full `RunnableSequence` — primitives are designed to nest inside one another.

Quick Revision — `RunnableParallel`

- `RunnableParallel` executes multiple `Runnables` concurrently on the same input.
- Constructed from a dictionary: `{key: Runnable}`.
- Every branch receives an identical copy of the input.
- Output is a dictionary keyed by the names given at construction.
- Branches can use entirely different models or providers.

4. `RunnablePassthrough`

Definition

`RunnablePassthrough` is a special `Runnable` primitive that returns whatever input it receives, completely unchanged — it performs no processing at all.

Why It Exists

In a purely sequential chain, only the *final* step's output is ever visible — intermediate results are discarded. Sometimes an intermediate value (such as the original joke, before it gets explained) needs to appear in the final output alongside a derived value. Rather than restructuring the whole chain, `RunnablePassthrough` is dropped into a `RunnableParallel` branch to preserve that intermediate value as-is.

How It Works

- Whatever is passed to `.invoke()` is returned identically — a string in, a string out; a dict in, the same dict out.

- It is almost always used as one branch inside a `RunnableParallel`, so that the same data reaching that branch is preserved untouched while other branches transform it.

Example — Preserving the Joke Alongside Its Explanation

Without a passthrough, a sequential joke → explanation chain would only ever print the explanation, since the joke text gets “consumed” as an intermediate step. Adding a `RunnableParallel` with one `RunnablePassthrough` branch keeps the joke visible in the final result.

```
from langchain_groq import ChatGroq
from langchain_core.runnables import RunnablePassthrough, RunnableSequence, RunnableParallel
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model = "llama-3.3-70b-versatile")

prompt1 = PromptTemplate(
    template = "Tell me a funny joke about the topic : {topic}",
    input_variables = ["topic"]
)

prompt2 = PromptTemplate(
    template = "Explain the joke in simple terms, joke : {joke}",
    input_variables = ["joke"]
)

parser = StrOutputParser()

joke_generator_chain = RunnableSequence(prompt1, model, parser)

# "joke" branch: passthrough returns the joke string unchanged.
# "explanation" branch: re-runs the joke chain, then explains it.
parallel_chain = RunnableParallel({
    "joke" : RunnablePassthrough(),
    "explanation" : RunnableSequence(joke_generator_chain, prompt2, model, parser)
})

final_chain = RunnableSequence(joke_generator_chain, parallel_chain)
output = final_chain.invoke({"topic": "Freedom Fighters"})
print(output)
```

`joke_generator_chain` produces the joke first. That joke string is then fed into `parallel_chain`, where the "joke" branch (a `RunnablePassthrough`) returns it unchanged, while the "explanation" branch reruns generation and then explains it. The final output is a single dict containing both the original joke and its explanation.

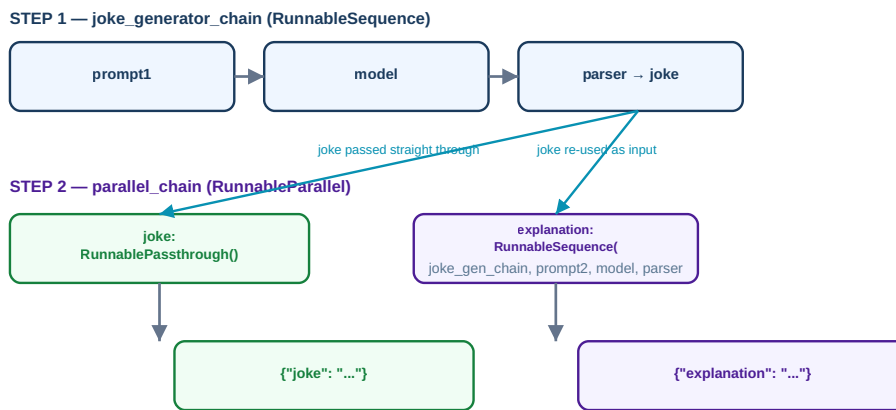


Figure 7.3 — `RunnablePassthrough` preserving the intermediate `joke` value alongside a derived explanation.

COMMON MISTAKE

Assuming a sequential chain automatically exposes every intermediate result. It does not — only the last `Runnable`'s output is returned unless a parallel branch with `RunnablePassthrough()` is used to retain an earlier value.

Quick Revision — `RunnablePassthrough`

- `RunnablePassthrough` returns its input unchanged, with zero processing.
- It solves the problem of losing intermediate values in a sequential chain.
- Almost always used as one branch inside a `RunnableParallel`.
- Useful whenever both a raw value and a derived value are needed in the final output.

5. `RunnableLambda`

Definition

`RunnableLambda` is a `Runnable` primitive that wraps any ordinary Python function (or a `lambda`), converting it into a full `Runnable` that can be composed with other `Runnables` in a chain.

Why It Exists

Real pipelines often need custom logic that no built-in component provides — text cleaning, word counting, format conversion, validation. `RunnableLambda` lets any such custom Python logic become a first-class citizen of a `LangChain` pipeline instead of living outside it.

How It Works

1. Write a normal Python function that accepts one input and returns a value.
2. Wrap it: `RunnableLambda(your_function)`.

3. The result is a Runnable with an `.invoke()` method and can be placed anywhere a Runnable is expected — including inside a `RunnableSequence` or as a branch of `RunnableParallel`.
4. A short `lambda` expression can be passed directly instead of a named function for simple one-line logic.

Example — Counting Words Without Asking the LLM

LLMs are often unreliable at precise counting tasks. Instead of asking the model how many words are in a joke, a plain Python function computes it deterministically, and `RunnableLambda` lets that function sit inside the same chain as the LLM call.

```
from langchain_groq import ChatGroq
from langchain_core.runnables import RunnablePassthrough, RunnableSequence,
RunnableParallel, RunnableLambda
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model = "llama-3.3-70b-versatile")

prompt = PromptTemplate(
    template = "Tell me a funny joke about the topic : {topic}",
    input_variables = ["topic"]
)

parser = StrOutputParser()

# Ordinary Python function -- not yet a Runnable.
def count_words(joke: str):
    return len(joke.split())

joke_gen_chain = RunnableSequence(prompt, model, parser)

# RunnableLambda wraps the plain function, making it composable.
parallel_chain = RunnableParallel(
    {
        "Word_count" : RunnableLambda(count_words),
        "Joke" : RunnablePassthrough()
    }
)

final_chain = RunnableSequence(joke_gen_chain, parallel_chain)

result = final_chain.invoke({"topic" : "Global Warming"})

print(result)
```

`joke_gen_chain` generates the joke text first. That text then reaches `parallel_chain`, where `"Word_count"` runs the wrapped `count_words` function on it, and `"Joke"` passes it through unchanged. Final result: `{"Word_count": N, "Joke": "..."}.`

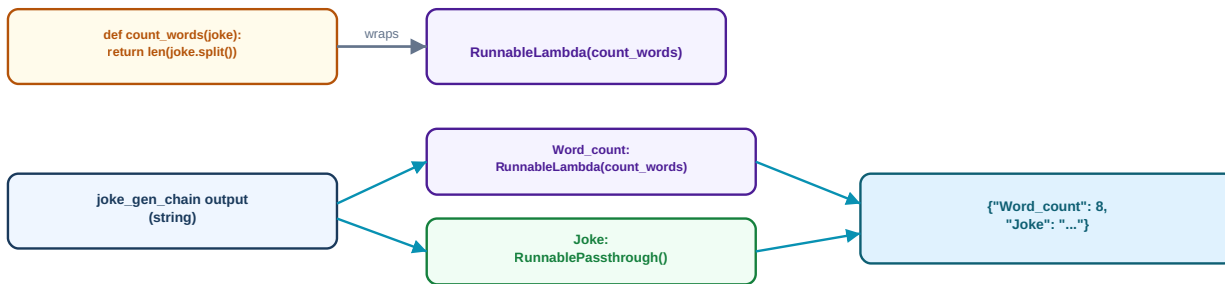


Figure 7.4 — A plain Python function wrapped by `RunnableLambda` and composed inside a `RunnableParallel`.

BEST PRACTICE

For simple one-line logic, pass a `lambda` directly — e.g. `RunnableLambda(lambda x: len(x.split()))` — instead of defining a named function; both produce identical results, and the inline form is what gives the primitive its name.

Quick Revision — `RunnableLambda`

- `RunnableLambda` converts any Python function into a `Runnable`.
- It bridges custom business logic with the LangChain pipeline interface.
- Accepts either a named function or an inline lambda expression.
- Commonly used for tasks LLMs are unreliable at, like precise counting.
- Once wrapped, it can be placed inside a `RunnableSequence` or `RunnableParallel` like any other `Runnable`.

6. `RunnableBranch`

Definition

`RunnableBranch` is a `Runnable` primitive that implements conditional routing — LangChain's equivalent of an `if / elif / else` statement for chains.

Why It Exists

Many real workflows must behave differently depending on the data itself. For example, a long report should be summarized, while a short one should be returned as-is; a support email might need to be routed differently depending on whether it is a complaint, a refund request, or a general query. `RunnableBranch` lets this decision live inside the pipeline itself.

How It Works

1. Construct `RunnableBranch` with one or more `(condition_function, runnable_to_execute)` tuples.
2. Each condition function receives the current input and must return `True` or `False`.

3. Conditions are evaluated in the order given; the first one that returns `True` triggers its paired `Runnable`.
4. A final, un-paired `Runnable` is supplied last as the default (“else”) case, executed only if every condition evaluated to `False`.

RUNNABLEBRANCH SYNTAX PATTERN

- 1 Open with `RunnableBranch(`
- 2 First tuple: `(condition_1, runnable_1)`
- 3 Additional tuples for each further condition, in order
- 4 Final entry: a bare `Runnable` with no condition — the default/else case

Example — Auto-Summarize Long Reports

The example generates a detailed report on a topic, then checks its word count. If the report is 500 words or longer it gets summarized by a second LLM call; otherwise it is returned unchanged via `RunnablePassthrough`.

```

from langchain_groq import ChatGroq
from langchain_core.runnables import RunnablePassthrough, RunnableSequence,
RunnableParallel, RunnableLambda, RunnableBranch
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model = "llama-3.3-70b-versatile")
parser = StrOutputParser()

prompt1 = PromptTemplate(
    template = "Write a detailed report about the topic : {topic}.",
    input_variables = ["topic"]
)

prompt2 = PromptTemplate(
    template = "Summarize the report: \n {report} \n in less than 500 words.",
    input_variables = ["report"]
)

report_gen_chain = RunnableSequence(prompt1, model, parser)

# RunnableBranch takes (condition, runnable) tuples, evaluated in order,
# plus one trailing default Runnable (acts as the "else").
conditional_chain = RunnableBranch(
    (lambda x: len(x.split()) >= 500, RunnableSequence(prompt2, model, parser)),
    RunnablePassthrough()
)

final_chain = RunnableSequence(report_gen_chain, conditional_chain)

result = final_chain.invoke({"topic" : "Global Warming"})

print(result)

```

`report_gen_chain` produces the full report string. That string is the input to `conditional_chain`: the lambda condition checks its word count; if `>= 500`, the report is summarized through `prompt2 → model → parser`; otherwise the default `RunnablePassthrough()` returns it untouched.

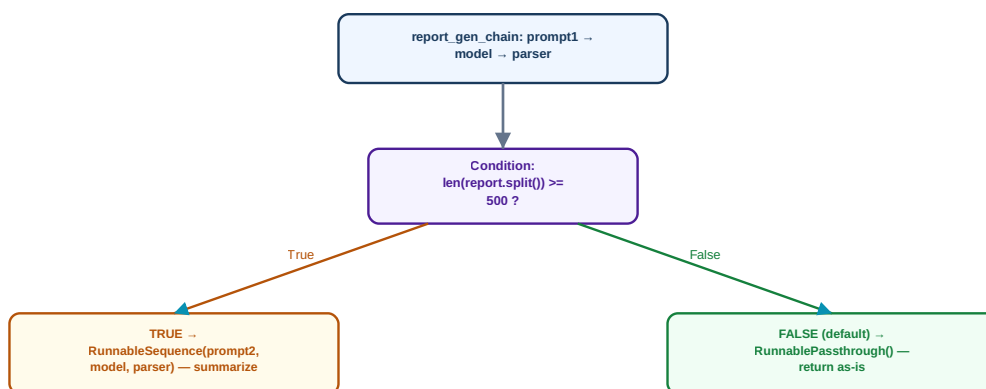


Figure 7.5 — `RunnableBranch` routing execution based on a word-count condition.

IMPORTANT

The trailing `Runnable` in `RunnableBranch` has no condition attached to it — it is always the default/else path, executed only when every preceding condition evaluates to `False`.

🔄 Quick Revision — `RunnableBranch`

- `RunnableBranch` is LangChain's conditional (if/elif/else) primitive.
- Built from (condition, runnable) tuples plus one trailing default `Runnable`.
- Conditions are plain functions (often lambdas) returning `True/False`.
- The first condition to return `True` determines which `Runnable` executes.
- The final, condition-less entry is the default/else case.

7. LCEL — LangChain Expression Language

Definition

LCEL (LangChain Expression Language) is a declarative syntax for building chains using Python's pipe operator `|`, as an alternative to explicitly instantiating `RunnableSequence`.

Why It Exists

Building a `RunnableSequence` by explicitly listing every `Runnable` is such a common pattern that the LangChain maintainers introduced a shorter, more readable way to express it: chaining `Runnables` with `|`, similar to a Unix pipe. This is considered a more declarative way of defining chains.

How It Works

Any two `Runnables` connected with `|` produce the exact same underlying `RunnableSequence` object as calling the class directly.

CLASS-BASED SYNTAX

```
chain = RunnableSequence(
    prompt1, model, parser
)
```

LCEL PIPE SYNTAX

```
chain = (
    prompt1 | model | parser
)
```

CLASS-BASED SYNTAX

`RunnableSequence(prompt1, model, parser)`

equivalent

LCEL PIPE SYNTAX (preferred going forward)

`prompt1 | model | parser`

Both produce the SAME `RunnableSequence` object. LCEL uses Python's `|` operator (overloaded via `__or__`) for a declarative, functional style. Currently `RunnableSequence` is the only primitive with pipe-syntax support in LangChain.

Figure 7.6 — Class-based `RunnableSequence` syntax versus LCEL pipe syntax — both produce an identical chain.

REAL WORLD INSIGHT

At present, LCEL's pipe syntax officially covers only **`RunnableSequence`**. There is no equivalent shorthand yet for `RunnableParallel`, `RunnableBranch`, or the other primitives — those still require their explicit class-based constructors. It is reasonable to expect LangChain to extend declarative syntax to more primitives in future versions.

Quick Revision — LCEL

- LCEL uses the `|` (pipe) operator to declaratively define chains.
- `prompt | model | parser` is functionally identical to `RunnableSequence(prompt, model, parser)`.
- LCEL currently applies only to sequential composition (`RunnableSequence`).
- It is the recommended, idiomatic way to write sequential chains going forward.
- Other primitives (`Parallel`, `Branch`, `Lambda`, `Passthrough`) still require their explicit class constructors.

K. Key Takeaways

- **Two Runnable categories** exist in LangChain: task-specific Runnables (Prompt, Model, Parser, Retriever) that do domain work, and Runnable primitives that orchestrate them.
- **RunnableSequence** chains any number of Runnables so each step's output feeds the next step's input.
- **RunnableParallel** runs multiple Runnables concurrently on identical input, returning a dictionary of results.
- **RunnablePassthrough** returns its input unchanged, and is used to preserve intermediate values that would otherwise be lost in a sequential chain.
- **RunnableLambda** converts any custom Python function (or lambda) into a composable Runnable.
- **RunnableBranch** implements conditional routing using (condition, Runnable) tuples plus a default case, acting as LangChain's if/elif/else.
- **LCEL** offers a declarative pipe-operator (`|`) shorthand for RunnableSequence, currently the only primitive with pipe-syntax support.
- All five primitives are themselves Runnables, so they can be freely nested inside one another to build arbitrarily complex AI workflows.

R. Revision Sheet

RunnableSequence

Chains Runnables to execute in order; each output becomes the next input. No limit on chain length.

RunnableParallel

Runs multiple Runnables concurrently on the same input; returns a dict keyed by branch name.

RunnablePassthrough

Returns its input exactly as received, with zero processing; preserves intermediate values inside a parallel branch.

RunnableLambda

Wraps any Python function or lambda so it becomes a composable Runnable inside a chain.

RunnableBranch

Conditional routing via (condition, Runnable) tuples plus a trailing default — LangChain's if/elif/else.

LCEL Pipe Syntax

`prompt | model | parser` is a declarative shorthand equivalent to `RunnableSequence`; currently limited to sequential chains.

I. Interview Questions

Q1. What is the difference between a task-specific Runnable and a Runnable primitive?

Q2. How does `RunnableSequence` determine what gets passed from one step to the next?

Q3. Why does `RunnableParallel` return a dictionary rather than a list or a single value?

Q4. Give a real scenario where `RunnablePassthrough` is necessary, and explain why a plain sequential chain would fail to meet that requirement.

Q5. How would you convert an arbitrary Python function into something usable inside a LangChain pipeline?

Q6. Explain the structure of a `RunnableBranch` call, including how the default case is defined.

Q7. What happens if none of the conditions in a `RunnableBranch` evaluate to `True` and no default `Runnable` is provided?

Q8. What is LCEL, and which primitive does its pipe syntax currently support?

Q9. Can `Runnable` primitives be nested inside one another? Justify your answer with an example.

Q10. Why might an AI Engineer prefer a plain Python function with `RunnableLambda` over asking an LLM to perform the same task (e.g. counting words)?

F. Further Reading

- LangChain Python documentation — `Runnable` interface and LCEL (python.langchain.com)
- LangChain Python documentation — `RunnableSequence`, `RunnableParallel`, `RunnableBranch`, `RunnableLambda` reference pages
- `langchain_groq` and `langchain_huggingface` integration package documentation